IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Datos listos para el modelo

Teoría, práctica y, al final, material de consulta

Sprint 6 · S06_03 · de consulta (no se proyecta) · cuaderno: 40 min en clase

IDEA GENERAL

Tres problemas, tres herramientas

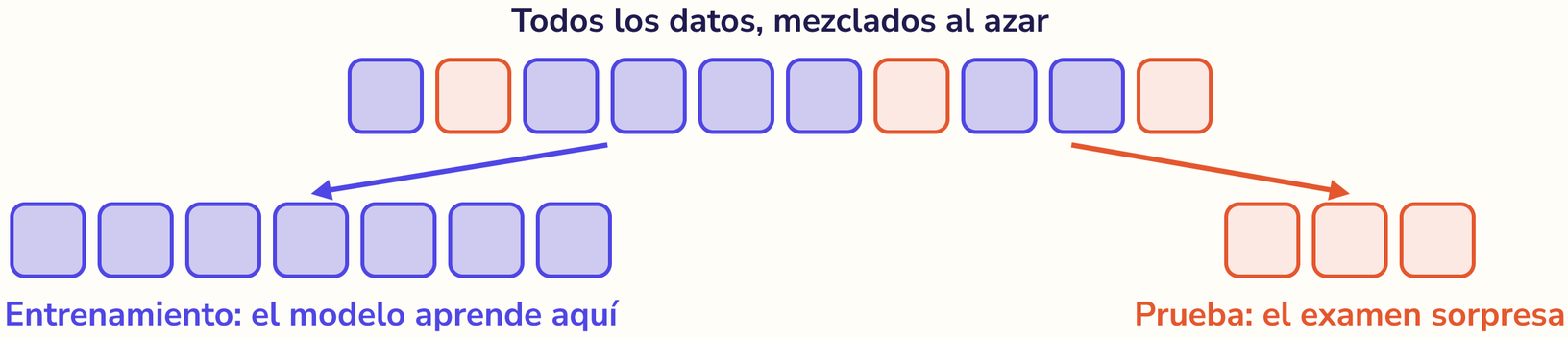
Preprocesar es dejar la tabla lista para el modelo: completa, en escalas parecidas y solo con números.

Hay huecos	`SimpleImputer`	**Imputar**: rellenar los vacíos, por ejemplo con la mediana.
Escalas distintas	`StandardScaler`	**Escalar**: un precio en euros pesa más que los niños de la reserva.
Texto, no números	`OneHotEncoder`	**Codificar**: convertir categorías, como la zona, en números.

SEPARAR ENTRENAMIENTO Y PRUEBA

Guardar un examen sorpresa

Si estudias con las preguntas del examen, un diez no demuestra nada: al modelo hay que **probarlo con datos que no ha visto**.



Un reparto justo

Al azar y con la misma proporción de cada respuesta (**estratificar**).

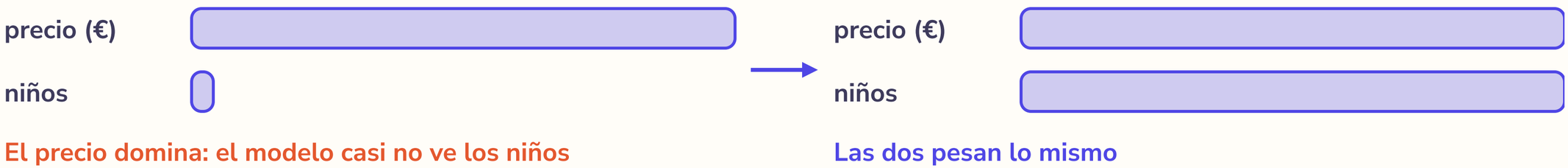
Una semilla

Un número que fija el azar: el mismo reparto cada vez.

ESCALADO

Poner las columnas en la misma escala

Muchos modelos se fijan en la columna de **números grandes**: escalar las pone en una escala comparable.



MinMaxScaler

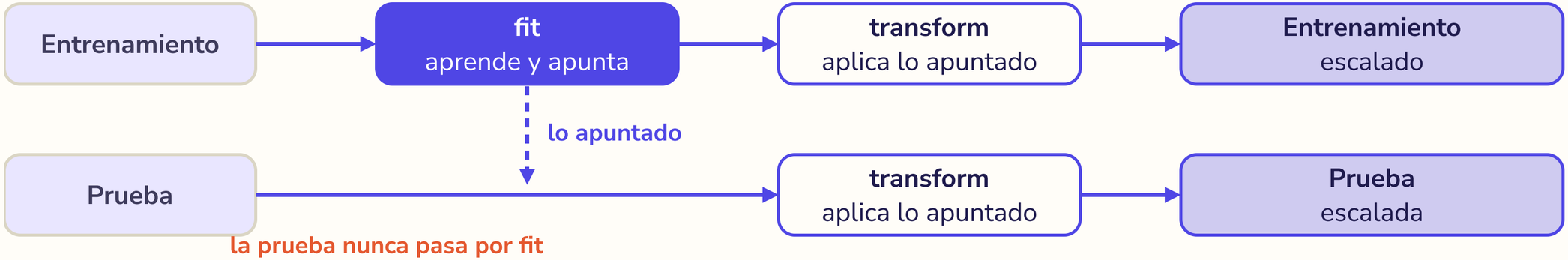
Todo entre **0 y 1**: (x – mínimo) / (máximo – mínimo).

StandardScaler

Media **0** y desviación **1**: (x – media) / desviación.

Cuidado con los extremos: un solo precio enorme aplasta a todos los demás con MinMax.

Aprender solo del entrenamiento



fit = aprender

Mira el entrenamiento y **apunta** lo que necesita.

transform = aplicar

Convierte valores, de entrenamiento **y de prueba**, con lo apuntado.

Regla: fit solo con entrenamiento. La prueba solo se transforma.

CODIFICACIÓN

Convertir categorías en números

Los modelos solo entienden números. Cómo codificar depende de si la categoría tiene **orden** o no.

Con orden → ordinal

satisfacción	código
baja	0
media	1
alta	2

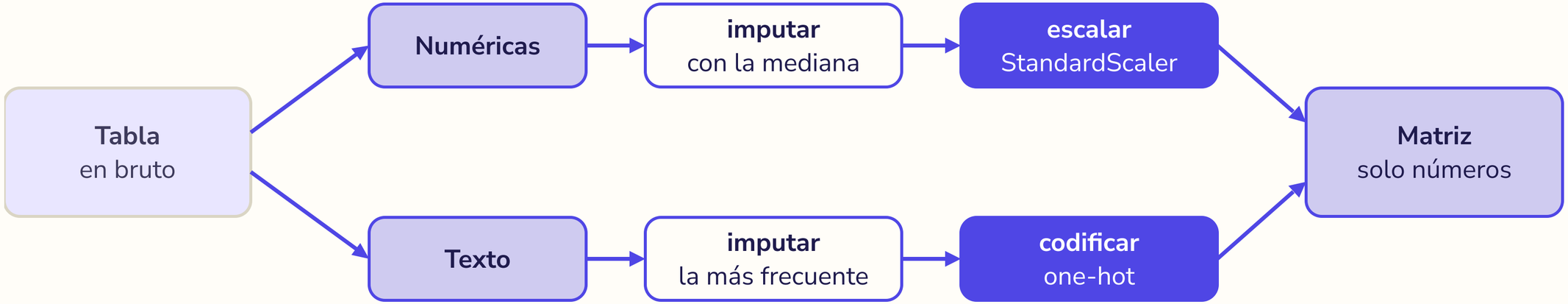
Sin orden → one-hot

zona	norte	oeste	sur
norte	1	0	0
sur	0	0	1
oeste	0	1	0

Un orden inventado engaña al modelo: con 1, 2 y 3, «sur» parecería «más» que «norte».

COLUMNTRANSFORMER

Cada tipo de columna, su receta



ColumnTransformer

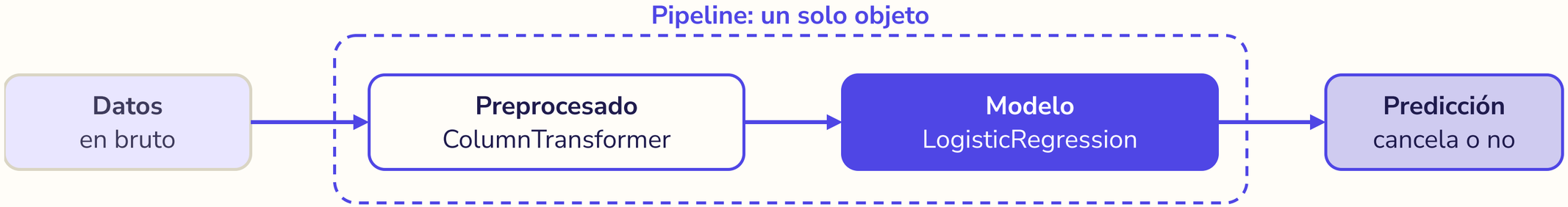
Reparte las columnas por **tipo**, aplica a cada grupo su receta y **junta** el resultado.

Cada receta, una cadena

Primero se **imputa** y luego se escala o se codifica. Las columnas que no se nombran se descartan.

PIPELINE

Una cadena de montaje para el modelo



fit

Cada paso aprende **solo** del entrenamiento y pasa el resultado al siguiente.

score

La prueba atraviesa los mismos pasos, **sin reajustar** nada. Devuelve la **exactitud**: proporción de aciertos.

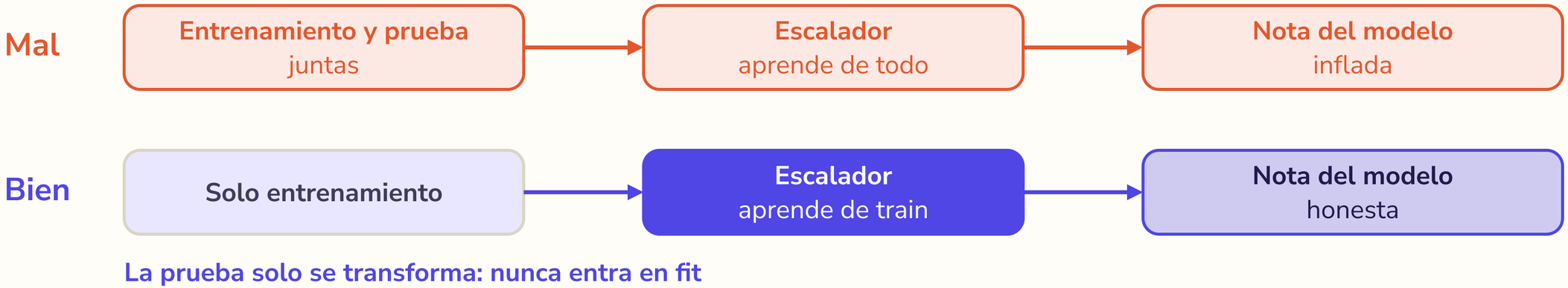
¿Es buena?

Se compara con una referencia, como decir siempre la respuesta **más frecuente**.

FUGA DE DATOS (CASA)

Cuando la prueba se cuela al aprender

Fuga de datos: el modelo «ve» información de la prueba mientras aprende, y su nota sale inflada.



Un caso típico: escalar o imputar **todo** el conjunto antes de separarlo. Un **Pipeline** lo evita: solo ajusta con lo que recibe **fit**.

REPRODUCIBILIDAD (CASA)

Mismos datos, mismas predicciones

Mismos datos + Misma configuración + Misma semilla = **Mismas predicciones**

Semilla fija

En el reparto y en el modelo: así el azar es el mismo cada vez.

Copia limpia

Para repetirlo se parte de una copia **sin entrenar** y con la misma configuración.

Por qué importa

Quien repita tu trabajo, en otro ordenador, debe obtener lo mismo.

PRÁCTICA · CUADERNO S06_03

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Secciones 1 y 2	Cargar las reservas limpias y definir **X** e **y**	Clase
Ejercicio 1	Separar entrenamiento y prueba	Casa
Sección 3	Imputar y escalar con **StandardScaler** (ejemplo)	Clase
Ejercicio 2	Escalar el precio con **MinMaxScaler**	Clase
Ejercicio 3	Codificar el depósito con orden (el canal en one-hot es ejemplo)	Casa
Ejercicio 4	Construir el **ColumnTransformer**	Clase
Ejercicio 5	**Pipeline** con el modelo y su exactitud en prueba	Clase
Secciones 6 y 7	Fuga de datos y un vistazo a PCA (ejemplos)	Casa
Mini-reto	Comprobar que el pipeline es reproducible	Casa

La «Solución de referencia» de cada ejercicio está plegada: se abre solo después de intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Después de intentar el ejercicio o si algo no te sale. No se proyecta en clase. Mismo **orden que el cuaderno**.

Ejercicio	Tema	Diapositivas
1 (casa)	Separar entrenamiento y prueba	13 y 14
Ejemplo 5 y 2	Escalar con los datos de entrenamiento	15 a 17
Ejemplo 7	Codificar el canal en one-hot	18 y 19
3 (casa)	Codificar el depósito con orden	20 y 21
4	Construir el ColumnTransformer	22 y 23
5	Pipeline con el modelo	24 y 25
Ejemplo 11 (casa)	Fuga de datos	26 y 27
Ejemplo 12 (casa)	Reducir columnas con PCA	28
Mini-reto (casa)	Pipeline reproducible	29 y 30

CONSULTA · SEPARAR ENTRENAMIENTO Y PRUEBA

Separar con train_test_split (ejercicio 1, casa)

fila	0	1	2	3	4	5	6	7	8	9
cancelada (y)	0	0	0	1	0	1	0	0	0	1
va a	train	**test**	train	train	train	train	**test**	train	train	**test**

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=7)
len(X_train), len(X_test)        # 7 3
```

Ejemplo con diez reservas de mentira. En el ejercicio 1, el 20 % para prueba y **random_state=SEMILLA**.

CONSULTA · ACLARACIONES · SEPARAR

Leer train_test_split (ejercicio 1, casa)

Devuelve cuatro cosas

En este orden: **X_train**, **X_test**, **y_train** e **y_test**. Hay que asignar las cuatro.

test_size=0.2

Reserva el 20 % para prueba: con 1.500 filas, **1.200** de entrenamiento y **300** de prueba.

stratify=y

Mantiene la proporción de canceladas en los dos trozos: **0,471** en train y **0,47** en test.

random_state=SEMILLA

Fija el reparto. Sin él, cada ejecución daría uno distinto y los resultados no serían **repetibles**.

CONSULTA · ESCALAR

Los dos escaladores con números (ejemplo 5)

Precio (€)	MinMax	Standard
60	0	−1,18
80	0,25	−0,51
100	0,5	0,17
140	1	1,52

MinMaxScaler

(x − mínimo) / (máximo − mínimo). Aquí, mínimo 60 y máximo 140.

StandardScaler

(x − media) / desviación. Aquí, media 95 y desviación 29,58.

Si entre los precios hay un 999 €, MinMax aplasta a los demás: 60, 80, 100 y 140 quedan entre 0 y 0,09.

CONSULTA · ESCALAR

Ajustar con train, aplicar a test (ejercicio 2)

```
# X_train: 60, 80, 100, 140   ·   X_test: 100, (vacío), 160
esc_minmax = MinMaxScaler().fit(X_train[["precio_noche"]])            # mínimo 60, máximo 140
mediana_precio = X_train["precio_noche"].median()                     # 90.0
esc_minmax.transform(X_test[["precio_noche"]].fillna(mediana_precio)) # 0.5   0.375   1.25
```

El 1,25 es normal: el 160 € supera lo visto al aprender. Con fit_transform en la prueba saldría 0,14 · 0 · 1: el escalador se reajustaría con ella.

CONSULTA · ACLARACIONES · ESCALAR

Leer el escalado (ejemplo 5 y ejercicio 2)

fit solo con train

esc_minmax aprendió mínimo **52,7** y máximo **231,78**; la mediana de train es **93,65** €.

Por qué se sale de 0–1

precio_test_esc va de **0,025** a **1,006**: en test hay un precio mayor que el máximo de train. No es un error.

Medias ≈ 0 en train

En el ejemplo 5, **StandardScaler** resta la media de train: las 9 columnas quedan en ≈ 0 en train y entre −0,1 y 0,16 en test.

fit_transform en test

Daría media 0 por construcción, pero con **sus propios** números: es fuga, y con una reserva suelta no se podría.

CONSULTA · CODIFICAR

Una columna por categoría (ejemplo 7)

```
oh = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
oh.fit_transform(df[["zona"]])                  # una fila por reserva, una columna por zona
oh.get_feature_names_out()                      # zona_norte zona_oeste zona_sur
oh.transform(pd.DataFrame({"zona": ["este"]}))  # 0, 0, 0 (zona nueva)
```

Tabla de la teoría

Con norte, sur, norte y oeste salen **tres columnas** de ceros y unos: el 1 marca la zona de cada reserva.

handle_unknown="ignore"

Si en la prueba aparece una zona nueva (este), todas sus columnas valen **0** en vez de dar error.

CONSULTA · ACLARACIONES · ONE-HOT

Leer OneHotEncoder (ejemplo 7)

fit aprende las categorías

onehot.fit(X_train[["canal"]]) mira solo train y apunta **qué canales existen**.

Cuatro columnas nuevas

canal_agencia, **canal_directo**, **canal_empresa** y **canal_web_intermediaria**: una por canal.

Por qué no 0, 1, 2, 3

Los canales no tienen orden: con números, **web_intermediaria** «valdría más» que **agencia**.

Canal nuevo en test

Lo codifica con **ceros** en todas las columnas y sigue; sin **handle_unknown="ignore"** daría un error.

CONSULTA · CODIFICAR

Codificar con orden (ejercicio 3, casa)

```
orden = [["baja", "media", "alta"]]   # una lista por columna
ord_sat = OrdinalEncoder(categories=orden)
ord_sat.fit_transform(df[["satisfaccion"]])   # media, alta, baja, media -> 1, 2, 0, 1
```

Sin categories, el orden sería alfabético: alta 0, baja 1, media 2. Un orden inventado.

Ejemplo con la columna de satisfacción de la teoría. En el ejercicio 3, el encoder se llama **ord_dep** y la columna es **deposito**.

CONSULTA · ACLARACIONES · ORDINAL

Leer OrdinalEncoder (ejercicio 3, casa)

categories=[[...]]

Una lista por columna y con el **orden real**: **sin_deposito**, **parcial**, **total**. De ahí los dobles corchetes.

Un número por depósito

sin_deposito 0, **parcial** 1 y **total** 2. En train: **667**, **361** y **172** reservas.

Sin categories

Orden alfabético: **parcial** 0, **sin_deposito** 1, **total** 2. Un orden falso: no pagar nada valdría más que pagar una parte.

total va el último

Si lo pones en otra posición, **total** no vale 2 y el ejercicio avisa: «'total' debe valer 2».

CONSULTA · COLUMNTRANSFORMER

Una receta por tipo de columna (ejercicio 4)

```
prep = ColumnTransformer([       # cada rama: (nombre, qué hacer, columnas)
    ("num", Pipeline([("imputar", SimpleImputer(strategy="median")),
                      ("escalar", StandardScaler())]), ["precio"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["zona"])])
# el precio vacío se rellena con la mediana (100) y al escalar queda en 0
```

precio	zona	num__precio	cat__zona_norte	cat__zona_oeste	cat__zona_sur
60	norte	−1,41	1	0	0
vacío	sur	0	0	0	1
100	norte	0	1	0	0
140	oeste	1,41	0	1	0

CONSULTA · ACLARACIONES · COLUMNTRANSFORMER

Leer el ColumnTransformer (ejercicio 4)

Una rama, tres datos

(nombre, receta, columnas): **"num"** va sobre **columnas_num** y **"cat"** sobre **columnas_cat**.

Cada rama es un **Pipeline**

Primero **imputar** (la mediana o la más frecuente) y después **escalar** o **codificar**.

34 columnas

9 numéricas + **25** de one-hot: los valores distintos de las 6 categóricas (4 + 4 + 4 + 8 + 2 + 3).

1.200 filas

fit_transform(X_train) solo transforma el entrenamiento: **1.200** filas, no las 1.500.

CONSULTA · PIPELINE

Pipeline con el modelo (ejercicio 5)

```
modelo = Pipeline([("prep", preprocesado),
                   ("clf", LogisticRegression(max_iter=1000, random_state=SEMILLA))])
modelo.fit(X_train, y_train)               # prep aprende SOLO de train; luego el clasificador
acc_test = modelo.score(X_test, y_test)    # exactitud: proporción de aciertos
```

fit

Cada paso aprende **solo** de entrenamiento y pasa el resultado al siguiente.

score

La prueba atraviesa los mismos pasos, **sin reajustar** nada, y devuelve la **exactitud**.

CONSULTA · ACLARACIONES · PIPELINE

Leer el pipeline (ejercicio 5)

fit ajusta todo

Con train se ajustan medianas, medias, categorías y la regresión logística. **Nada de test** interviene.

score acepta datos «crudos»

El pipeline aplica por dentro el preprocesado, con **transform**, antes de predecir.

Exactitud 0,680

Frente a **0,530** de «decir siempre la clase mayoritaria»: el modelo aporta **15 puntos**.

Si no sale

Si la exactitud no supera 0,60, revisa que ajustas con train, puntúas con test y que **preprocesado** está dentro.

CONSULTA · FUGA DE DATOS (CASA)

Escalar con todo o solo con train (ejemplo 11)

8 precios de ejemplo: entrenamiento 60, 80, 100, 100, 120, 140 · prueba 180 y 300.

MAL: FIT CON TODOS

```
esc = StandardScaler().fit(todos)
esc.mean_[0]              # 135.0
esc.transform([[300]])    # 2.32
```

BIEN: FIT SOLO CON TRAIN

```
esc = StandardScaler().fit(train)
esc.mean_[0]              # 100.0
esc.transform([[300]])    # 7.75
```

El escalador «mal» ya conocía el 300 y lo ve casi normal (2,3 desviaciones). **Pipeline** lo evita: solo ajusta con lo que recibe **fit**.

CONSULTA · ACLARACIONES · FUGA DE DATOS

Leer el ejemplo de la fuga (ejemplo 11, casa)

esc_mal

Aprende la media del precio con **todas** las reservas, también las de test: **102,179**.

esc_bien

La aprende solo con **X_train**: **102,144**. En los dos casos, los nulos se rellenan antes con la mediana correspondiente.

Casi iguales

Se diferencian en **0,035 €**: con un escalado el efecto de la fuga es muy pequeño.

Importa igual

Es un mal hábito: con otras transformaciones puede **inflar mucho** las métricas, y en producción no existirá un «todo». Con **Pipeline** desaparece.

CONSULTA · AMPLIACIÓN · PCA (CASA)

Resumir muchas columnas con PCA (ejemplo 12)

```
pca = PCA(n_components=5, random_state=SEMILLA).fit(num_train)
pca.explained_variance_ratio_    # ≈ 0,178 · 0,126 · 0,118 · 0,114 · 0,112
```

Qué hace

Resume muchas columnas en pocas **componentes**, las que más variación recogen. Solo con columnas numéricas y ya escaladas.

Qué sale

La primera explica ≈ **18 %**; las cinco juntas, ≈ **65 %** de la variación de las 9 columnas.

Barras parecidas

Las columnas apenas están **correlacionadas**: no hay una dirección que concentre la variación y PCA no puede resumir mucho.

CONSULTA · REPRODUCIBILIDAD (CASA)

Comprobar que es reproducible (mini-reto)

```
modelo2 = clone(modelo)                  # copia nueva, sin entrenar
modelo2.fit(X_train, y_train)
pred1 = modelo.predict(X_test)
pred2 = modelo2.predict(X_test)
np.array_equal(pred1, pred2)             # True: predicciones idénticas
```

clone

Crea un pipeline **nuevo y sin entrenar**, con la misma configuración.

np.array_equal

Compara dos listas: **True** si son **idénticas**.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la comprobación (mini-reto, casa)

clone(modelo)

Copia la **configuración**, no lo aprendido: hay que volver a ajustarlo.

son_iguales vale True

Mismos datos, misma configuración y semilla fija: el mismo ajuste y **las mismas predicciones**.

No basta modelo2 = modelo

Es el **mismo objeto** con otro nombre: compararlo consigo mismo no demuestra nada.

Si sale False

Alguna parte aleatoria no tiene **random_state=SEMILLA**, o has ajustado con datos distintos.